

Deep Learning — Assignment 4

Fourth assignment for the 2023 Deep Learning course (NWI-IMC070) of the Radboud University.

Names: Vojko Pjanovic (s1084416); Carmen Gascó Gálvez (s1098566)

Group: 10

Instructions:

- Fill in your names and the name of your group.
- Answer the questions and complete the code where necessary.
- Keep your answers brief, one or two sentences is usually enough.
- Re-run the whole notebook before you submit your work.
- Save the notebook as a PDF and submit that in Brightspace together with the `.ipynb` notebook file.
- The easiest way to make a PDF of your notebook is via File > Print Preview and then use your browser's print option to print to PDF.

Note:

- The models in this assignment take a while to train. It is faster on a GPU (e.g., on Google Colab), but still doable on a CPU. Plan ahead to leave enough time to analyse your results.

Objectives

In this assignment you will

1. Implement an LSTM module from scratch.
2. Use the built-in LSTM module from PyTorch.
3. Compare fully connected and recurrent neural networks in an experiment.
4. Experiment with data augmentation.

Required software

If you haven't done so already, you will need to install the following additional libraries:

- `torch` and `torchvision` for PyTorch,
- `d2l`, the library that comes with [Dive into deep learning](#) book.

All libraries can be installed with `pip install`.

```
In [ ]: %matplotlib inline
import csv
import glob
import re
import numpy as np
import scipy.io
import scipy.signal
import matplotlib.pyplot as plt
import torch
!pip install d2l==1.0.0a1.post0
from d2l import torch as d2l

# fix the seed, so outputs are exactly reproducible
torch.manual_seed(12345);
```

Requirement already satisfied: d2l==1.0.0a1.post0 in /usr/local/lib/python3.10/dist-packages (1.0.0a1.post0)

Requirement already satisfied: jupyter in /usr/local/lib/python3.10/dist-packages (from d2l==1.0.0a1.post0) (1.0.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from d2l==1.0.0a1.post0) (1.23.5)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from d2l==1.0.0a1.post0) (3.7.1)

Requirement already satisfied: matplotlib-inline in /usr/local/lib/python3.10/dist-packages (from d2l==1.0.0a1.post0) (0.1.6)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from d2l==1.0.0a1.post0) (2.31.0)

Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from d2l==1.0.0a1.post0) (1.5.3)

Requirement already satisfied: gym in /usr/local/lib/python3.10/dist-packages (from d2l==1.0.0a1.post0) (0.25.2)

Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from gym->d2l==1.0.0a1.post0) (2.2.1)

Requirement already satisfied: gym-notices>=0.0.4 in /usr/local/lib/python3.10/dist-packages (from gym->d2l==1.0.0a1.post0) (0.0.8)

Requirement already satisfied: notebook in /usr/local/lib/python3.10/dist-packages (from jupyter->d2l==1.0.0a1.post0) (6.5.5)

Requirement already satisfied: qtconsole in /usr/local/lib/python3.10/dist-packages (from jupyter->d2l==1.0.0a1.post0) (5.4.4)

Requirement already satisfied: jupyter-console in /usr/local/lib/python3.10/dist-packages (from jupyter->d2l==1.0.0a1.post0) (6.1.0)

Requirement already satisfied: nbconvert in /usr/local/lib/python3.10/dist-packages (from jupyter->d2l==1.0.0a1.post0) (6.5.4)

Requirement already satisfied: ipykernel in /usr/local/lib/python3.10/dist-packages (from jupyter->d2l==1.0.0a1.post0) (5.5.6)

Requirement already satisfied: ipywidgets in /usr/local/lib/python3.10/dist-packages (from jupyter->d2l==1.0.0a1.post0) (7.7.1)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (1.1.0)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (4.42.1)

Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (1.4.5)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (23.1)

Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (9.4.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (3.1.1)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib->d2l==1.0.0a1.post0) (2.8.2)

Requirement already satisfied: traitlets in /usr/local/lib/python3.10/dist-packages (from matplotlib-inline->d2l==1.0.0a1.post0) (5.7.1)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->d2l==1.0.0a1.post0) (2023.3.post1)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->d2l==1.0.0a1.post0) (3.2.0)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->d2l==1.0.0a1.post0) (3.4)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->d2l==1.0.0a1.post0) (2.0.4)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->d2l==1.0.0a1.post0) (2023.7.22)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.7->matplotlib->d2l==1.0.0a1.post0) (1.16.0)

Requirement already satisfied: ipython-genutils in /usr/local/lib/python3.10

0/dist-packages (from ipykernel->jupyter->d2l==1.0.0a1.post0) (0.2.0)
Requirement already satisfied: ipython>=5.0.0 in /usr/local/lib/python3.10/dist-packages (from ipykernel->jupyter->d2l==1.0.0a1.post0) (7.34.0)
Requirement already satisfied: jupyter-client in /usr/local/lib/python3.10/dist-packages (from ipykernel->jupyter->d2l==1.0.0a1.post0) (6.1.12)
Requirement already satisfied: tornado>=4.2 in /usr/local/lib/python3.10/dist-packages (from ipykernel->jupyter->d2l==1.0.0a1.post0) (6.3.2)
Requirement already satisfied: widgetsnbextension~=3.6.0 in /usr/local/lib/python3.10/dist-packages (from ipywidgets->jupyter->d2l==1.0.0a1.post0) (3.6.5)
Requirement already satisfied: jupyterlab-widgets>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from ipywidgets->jupyter->d2l==1.0.0a1.post0) (3.0.8)
Requirement already satisfied: prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from jupyter-console->jupyter->d2l==1.0.0a1.post0) (3.0.39)
Requirement already satisfied: pygments in /usr/local/lib/python3.10/dist-packages (from jupyter-console->jupyter->d2l==1.0.0a1.post0) (2.16.1)
Requirement already satisfied: lxml in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (4.9.3)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (4.11.2)
Requirement already satisfied: bleach in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (6.0.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (0.7.1)
Requirement already satisfied: entrypoints>=0.2.2 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (0.4)
Requirement already satisfied: Jinja2>=3.0 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (3.1.2)
Requirement already satisfied: jupyter-core>=4.7 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (5.3.1)
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (0.2.2)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (2.1.3)
Requirement already satisfied: mistune<2,>=0.8.1 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (0.8.4)
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (0.8.0)
Requirement already satisfied: nbformat>=5.1 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (5.9.2)
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (1.5.0)
Requirement already satisfied: tinycss2 in /usr/local/lib/python3.10/dist-packages (from nbconvert->jupyter->d2l==1.0.0a1.post0) (1.2.1)
Requirement already satisfied: pyzmq<25,>=17 in /usr/local/lib/python3.10/dist-packages (from notebook->jupyter->d2l==1.0.0a1.post0) (23.2.1)
Requirement already satisfied: argon2-cffi in /usr/local/lib/python3.10/dist-packages (from notebook->jupyter->d2l==1.0.0a1.post0) (23.1.0)
Requirement already satisfied: nest-asyncio>=1.5 in /usr/local/lib/python3.10/dist-packages (from notebook->jupyter->d2l==1.0.0a1.post0) (1.5.7)
Requirement already satisfied: Send2Trash>=1.8.0 in /usr/local/lib/python3.10/dist-packages (from notebook->jupyter->d2l==1.0.0a1.post0) (1.8.2)
Requirement already satisfied: terminado>=0.8.3 in /usr/local/lib/python3.10/dist-packages (from notebook->jupyter->d2l==1.0.0a1.post0) (0.17.1)
Requirement already satisfied: prometheus-client in /usr/local/lib/python3.10/dist-packages (from notebook->jupyter->d2l==1.0.0a1.post0) (0.17.1)
Requirement already satisfied: nbclassic>=0.4.7 in /usr/local/lib/python3.10/dist-packages (from notebook->jupyter->d2l==1.0.0a1.post0) (1.0.0)
Requirement already satisfied: qtpy>=2.4.0 in /usr/local/lib/python3.10/dist-packages (from qtconsole->jupyter->d2l==1.0.0a1.post0) (2.4.0)
Requirement already satisfied: setuptools>=18.5 in /usr/local/lib/python3.10/dist-packages (from ipython>=5.0.0->ipykernel->jupyter->d2l==1.0.0a1.post0)

0) (67.7.2)

Requirement already satisfied: jedi>=0.16 in /usr/local/lib/python3.10/dist-packages (from ipython>=5.0.0->ipykernel->jupyter->d2l==1.0.0a1.post0) (0.19.0)

Requirement already satisfied: decorator in /usr/local/lib/python3.10/dist-packages (from ipython>=5.0.0->ipykernel->jupyter->d2l==1.0.0a1.post0) (4.4.2)

Requirement already satisfied: pickleshare in /usr/local/lib/python3.10/dist-packages (from ipython>=5.0.0->ipykernel->jupyter->d2l==1.0.0a1.post0) (0.7.5)

Requirement already satisfied: backcall in /usr/local/lib/python3.10/dist-packages (from ipython>=5.0.0->ipykernel->jupyter->d2l==1.0.0a1.post0) (0.2.0)

Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.10/dist-packages (from ipython>=5.0.0->ipykernel->jupyter->d2l==1.0.0a1.post0) (4.8.0)

Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.10/dist-packages (from jupyter-core>=4.7->nbconvert->jupyter->d2l==1.0.0a1.post0) (3.10.0)

Requirement already satisfied: jupyter-server>=1.8 in /usr/local/lib/python3.10/dist-packages (from nbclassic>=0.4.7->notebook->jupyter->d2l==1.0.0a1.post0) (1.24.0)

Requirement already satisfied: notebook-shim>=0.2.3 in /usr/local/lib/python3.10/dist-packages (from nbclassic>=0.4.7->notebook->jupyter->d2l==1.0.0a1.post0) (0.2.3)

Requirement already satisfied: fastjsonschema in /usr/local/lib/python3.10/dist-packages (from nbformat>=5.1->nbconvert->jupyter->d2l==1.0.0a1.post0) (2.18.0)

Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.10/dist-packages (from nbformat>=5.1->nbconvert->jupyter->d2l==1.0.0a1.post0) (4.19.0)

Requirement already satisfied: wcwidth in /usr/local/lib/python3.10/dist-packages (from prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0->jupyter-console->jupyter->d2l==1.0.0a1.post0) (0.2.6)

Requirement already satisfied: ptyprocess in /usr/local/lib/python3.10/dist-packages (from terminado>=0.8.3->notebook->jupyter->d2l==1.0.0a1.post0) (0.7.0)

Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.10/dist-packages (from argon2-cffi->notebook->jupyter->d2l==1.0.0a1.post0) (21.2.0)

Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.10/dist-packages (from beautifulsoup4->nbconvert->jupyter->d2l==1.0.0a1.post0) (2.5)

Requirement already satisfied: webencodings in /usr/local/lib/python3.10/dist-packages (from bleach->nbconvert->jupyter->d2l==1.0.0a1.post0) (0.5.1)

Requirement already satisfied: parso<0.9.0,>=0.8.3 in /usr/local/lib/python3.10/dist-packages (from jedi>=0.16->ipython>=5.0.0->ipykernel->jupyter->d2l==1.0.0a1.post0) (0.8.3)

Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.10/dist-packages (from jsonschema>=2.6->nbformat>=5.1->nbconvert->jupyter->d2l==1.0.0a1.post0) (23.1.0)

Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.10/dist-packages (from jsonschema>=2.6->nbformat>=5.1->nbconvert->jupyter->d2l==1.0.0a1.post0) (2023.7.1)

Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.10/dist-packages (from jsonschema>=2.6->nbformat>=5.1->nbconvert->jupyter->d2l==1.0.0a1.post0) (0.30.2)

Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.10/dist-packages (from jsonschema>=2.6->nbformat>=5.1->nbconvert->jupyter->d2l==1.0.0a1.post0) (0.10.2)

Requirement already satisfied: anyio<4,>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from jupyter-server>=1.8->nbclassic>=0.4.7->notebook->jupyter->d2l==1.0.0a1.post0) (3.7.1)

Requirement already satisfied: websocket-client in /usr/local/lib/python3.10

```

0/dist-packages (from jupyter-server>=1.8->nbclassic>=0.4.7->notebook->jupyter->d2l==1.0.0a1.post0) (1.6.2)
Requirement already satisfied: cffi>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from argon2-cffi-bindings->argon2-cffi->notebook->jupyter->d2l==1.0.0a1.post0) (1.15.1)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.10/dist-packages (from anyio<4,>=3.1.0->jupyter-server>=1.8->nbclassic>=0.4.7->notebook->jupyter->d2l==1.0.0a1.post0) (1.3.0)
Requirement already satisfied: exceptiongroup in /usr/local/lib/python3.10/dist-packages (from anyio<4,>=3.1.0->jupyter-server>=1.8->nbclassic>=0.4.7->notebook->jupyter->d2l==1.0.0a1.post0) (1.1.3)
Requirement already satisfied: pycparser in /usr/local/lib/python3.10/dist-packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->notebook->jupyter->d2l==1.0.0a1.post0) (2.21)

```

4.1 Dataset: Atrial fibrillation classification on ECG recordings (1 point)

In this assignment we will work with data from the [PhysioNet Computing in Cardiology Challenge 2017](#) to classify atrial fibrillation in electrocardiograms (ECGs). Atrial fibrillation is an abnormal heart rhythm, which can be recognized as irregular patterns in ECG recordings.

(a) Download the [training dataset](#) from the challenge website and extract the files.

```

In [ ]: !mkdir -p data
        !wget -c -O data/training2017.zip https://physionet.org/files/challenge-2017/1.0.0/training2017.zip
        !cd data/ ; unzip -qo training2017.zip

--2023-09-29 10:40:59-- https://physionet.org/files/challenge-2017/1.0.0/training2017.zip
Resolving physionet.org (physionet.org)... 18.18.42.54
Connecting to physionet.org (physionet.org)|18.18.42.54|:443... connected.
HTTP request sent, awaiting response... 416 Requested Range Not Satisfiable

```

The file is already fully retrieved; nothing to do.

The dataset consists of a number of recordings and corresponding labels. We use a subset of the dataset that includes only the samples with a normal rhythm (label N or class 0) and those with atrial fibrillation (label A or class 1).

(b) Run the code to load the data.

```

In [ ]: class ECGDataset(torch.utils.data.Dataset):
        # labels: 'N', 'A', 'O'
        def __init__(self, directory, max_length=18286, class_labels=('N', 'A', 'O')):
            super().__init__()
            self.class_labels = class_labels
            self.load_data(directory, max_length)

        def load_data(self, directory, max_length):
            label_map = {}
            with open('%s/REFERENCE.csv' % directory, 'r') as f:
                for line in csv.reader(f):
                    label_map[line[0]] = line[1]

            samples = []
            lengths = []

```

```

labels = []

for file in sorted(glob.glob('%s/*.mat' % directory)):
    subject_id = re.match('.(A[0-9]+)\.mat', file)[1]
    label = label_map[subject_id]
    if label not in self.class_labels:
        # skip this label
        continue
    mat_data = scipy.io.loadmat(file)
    sample = mat_data['val'][0]
    if len(sample) < 4000:
        # skip short samples
        continue
    samples.append(np.pad(sample, (0, max_length - len(sample))))
    lengths.append(len(sample))
    labels.append(self.class_labels.index(label_map[subject_id]))

# concatenate
samples = np.vstack(samples)
lengths = np.stack(lengths)
labels = np.stack(labels)

# convert to PyTorch tensors
self.samples = torch.tensor(samples, dtype=torch.float32)
self.lengths = torch.tensor(lengths, dtype=torch.long)
self.labels = torch.tensor(labels, dtype=torch.long)

@property
def class_proportions(self):
    return torch.mean((torch.arange(len(self.class_labels))[None, :] ==
                             self.labels[:, None]).to(torch.float), axis=0)

def __getitem__(self, index):
    l = self.lengths[index]
    x = self.samples[index, :l]
    y = self.labels[index]
    return x, y

def __len__(self):
    return self.samples.shape[0]

data = ECGDataset('data/training2017', class_labels=('N', 'A'))

```

The recordings have different lengths (between 30 to 60 seconds). There are more "normal" recordings than recordings that show atrial fibrillation.

(c) Print some statistics of the data.

```

In [ ]: print('Number of examples: %d' % len(data))
        print()
        print('Minimum length: %d' % torch.min(data.lengths))
        print('Median length: %d' % torch.median(data.lengths))
        print('Maximum length: %d' % torch.max(data.lengths))
        print()
        print('Class distribution:', data.class_proportions.numpy())

```

Number of examples: 5622

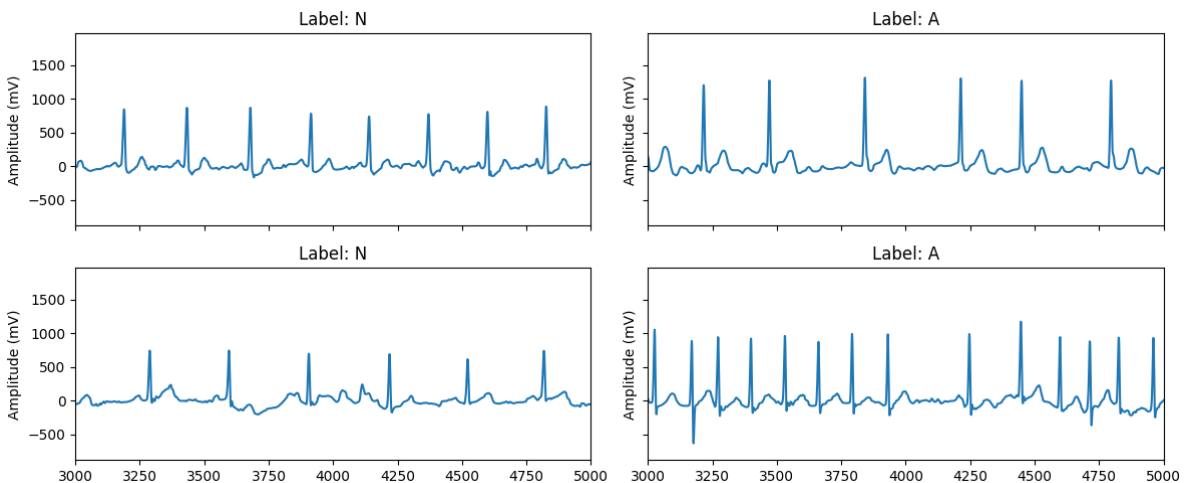
Minimum length: 4004
Median length: 9000
Maximum length: 18286

Class distribution: [0.87709 0.12290999]

Each example has a 1D vector that represents the ECG measurement over time.

(d) Run the code to plot two recordings from each class.

```
In [ ]: fig, axes = plt.subplots(nrows=2, ncols=2, sharex=True, sharey=True, figsize=
for i, idx in enumerate([0, 3, 1, 4]):
    x, y = data[idx]
    ax = axes[i // 2][i % 2]
    ax.plot(x)
    ax.set_title('Label: %s' % data.class_labels[y])
    ax.set_xlim(3000, 5000)
    ax.set_ylabel('Time (frames)')
    ax.set_ylabel('Amplitude (mV)')
plt.tight_layout()
```



(e) The class distribution in this dataset is quite unbalanced. What consequences could this have? (1 point)

In the case of having an unbalanced dataset would lead to have a model that is biased towards the majority class. This means that the model would be more likely to predict the normal rhythm. This would lead to a model that is not able to predict the minority class, which is the atrial fibrillation. This would be a problem because we would not be able to detect the atrial fibrillation in the ECG recordings.

4.2 Feature extraction

To simplify our problem a bit, we will convert the 1D ECG signals to [spectrograms](#). A spectrogram is a summary of the frequencies in small windows of the recording. These features will make it easier to train a classification model.

(a) Run the code to compute the spectrograms.


```
In [ ]: class ECGSpectrumDataset(ECGDataset):
    NPERSEG = 32
    NOVERLAP = 32 // 8

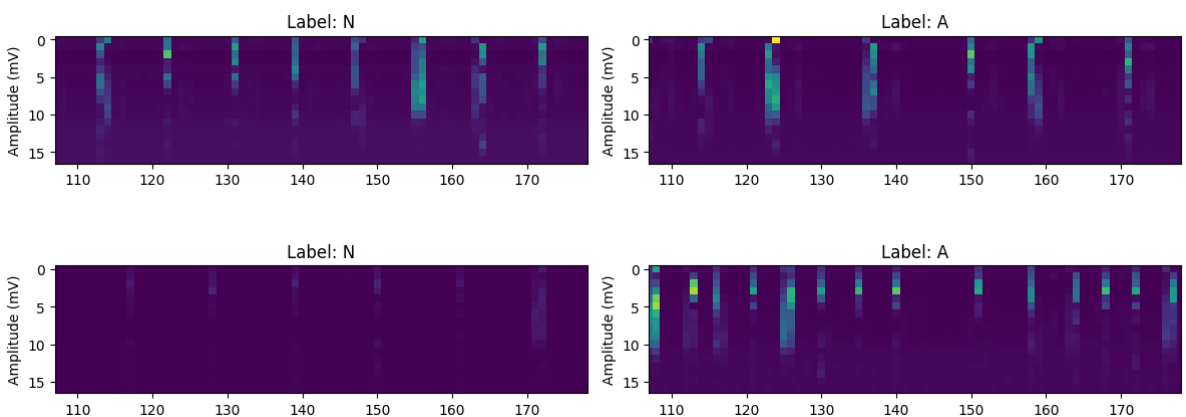
    def __init__(self, *args, **kwargs):
        # initialize the original dataset to load the samples
        super().__init__(*args, **kwargs)
        # compute and store the spectrograms to replace the samples
        self.compute_spectrum()

    def compute_spectrum(self):
        """
        Replaces the samples in this dataset with spectrograms.
        """
        f, t, Sxx = scipy.signal.spectrogram(self.samples.numpy(), scaling=
                                                nperseg=self.NPERSEG, noverlap=
                                                # normalize the measurements for each frequency
        Sxx = Sxx - np.mean(Sxx, axis=(0, 2), keepdims=True)
        Sxx = Sxx / np.std(Sxx, axis=(0, 2), keepdims=True)
        # replace the existing samples in the dataset with the computed spe
        self.samples = torch.tensor(Sxx.transpose(0, 2, 1))
        # recompute the length of each samples to account for the number of
        self.lengths = (self.lengths - self.NPERSEG) // (self.NPERSEG - sel

data_spectrum = ECGSpectrumDataset('data/training2017', class_labels=('N',
```

(b) Plot the spectrograms for the four samples from the previous plot.

```
In [ ]: plt.figure(figsize=(12, 5))
for i, idx in enumerate([0, 3, 1, 4]):
    x, y = data_spectrum[idx]
    plt.subplot(2, 2, i + 1)
    plt.imshow(x.T)
    plt.title('Label: %s' % data.class_labels[y])
    # show roughly the same segments as in the previous plot
    plt.xlim(3000 // 28, 5000 // 28)
    plt.ylabel('Amplitude (mV)')
plt.tight_layout()
```



The spectrogram data has 17 frequency bins for each window. We will use these as our input features. We normalized the data for each frequency to zero mean, unit variance.

(c) Print the statistics of the spectrum dataset and check the shape of the first sample.

```
In [ ]: print('Minimum length: %d' % torch.min(data_spectrum.lengths))
print('Median length: %d' % torch.median(data_spectrum.lengths))
```

```
print('Maximum length: %d' % torch.max(data_spectrum.lengths))
print()
print('Mean value: %f' % torch.mean(data_spectrum.samples))
print('Standard deviation: %f' % torch.std(data_spectrum.samples))
print()
# print the shape of the first sample
x, y = data_spectrum[0]
print('Shape of first sample:', x.shape)
```

Minimum length: 141
 Median length: 320
 Maximum length: 651

Mean value: 0.000327
 Standard deviation: 1.003635

Shape of first sample: torch.Size([320, 17])

4.3 Splitting training and validation sets

We will split our dataset in separate training and validation sets (80% – 20%).

(a) Run the code to create a random split.

```
In [ ]: train_samples = int(0.8 * len(data))
val_samples = len(data) - train_samples
data_train_original, data_val_original = torch.utils.data.random_split(data,
                                [train_samples, val_samples])

print('data_train:', len(data_train_original))
print('data_val: ', len(data_val_original))
```

data_train: 4497
 data_val: 1125

4.4 Creating a balanced dataset by resampling

As you have seen, the dataset contains far more normal recordings than recordings with atrial fibrillation. We will create a balanced dataset by including multiple copies of the atrial fibrillation samples.

In this assignment we will also use a balanced validation set. This is something you may or may not want to do in practice, because it means that your validation set is no longer representative of the test data. The advantage is that the accuracy on a balanced validation set is easier to compare with the accuracy on the training set.

(a) Run the code to create balanced training and validation sets.

```
In [ ]: def balance_dataset(dataset):
    # collect labels from the source dataset
    labels = torch.zeros((len(dataset),), dtype=torch.long)
    for i, (x, y) in enumerate(dataset):
        labels[i] = y
    indices = torch.arange(len(dataset), dtype=torch.long)

    unique_labels = np.unique(labels.numpy())

    # count the number of samples per class
    n = [torch.sum((labels == label).to(torch.long)).item() for label in unique_labels]
```

```

        for label in unique_labels]

    # perhaps the dataset is already balanced?
    if len(np.unique(n)) == 1:
        return dataset

    print('Samples per class:', n)
    for i, label in enumerate(unique_labels):
        # we will add more samples unless every class has the same number o
        while n[i] < max(n):
            extra_samples = max(n) - n[i]
            print('- Repeating %d samples for class %d' % (extra_samples, l

                # take a random subset of samples from this class
                idxs = torch.where(labels == label)[0]
                idxs = idxs[torch.randperm(idxs.shape[0])]
                idxs = idxs[:extra_samples]

                # add these indices to the list
                indices = torch.cat((indices, idxs))
                n[i] += len(idxs)

        # return the subset as a new torch dataset
        return torch.utils.data.Subset(dataset, indices)

    print('Balancing the training set')
    data_train = balance_dataset(data_train_original)
    print('Balancing the validation set')
    data_val = balance_dataset(data_val_original)

```

Balancing the training set

Samples per class: [3952, 545]

- Repeating 3407 samples for class 1
- Repeating 2862 samples for class 1
- Repeating 2317 samples for class 1
- Repeating 1772 samples for class 1
- Repeating 1227 samples for class 1
- Repeating 682 samples for class 1
- Repeating 137 samples for class 1

Balancing the validation set

Samples per class: [979, 146]

- Repeating 833 samples for class 1
- Repeating 687 samples for class 1
- Repeating 541 samples for class 1
- Repeating 395 samples for class 1
- Repeating 249 samples for class 1
- Repeating 103 samples for class 1

4.5 Splitting recordings into chunks

The recordings in our dataset all have different lengths and are generally quite long. To simplify training, we will split them into smaller chunks of 40 time steps each. This means that each recording will have multiple chunks in the dataset.

(a) Run the code to create the pre-chunked dataset.

```

In [ ]: class ChunkedDataset(torch.utils.data.TensorDataset):
        def __init__(self, source_dataset, chunk_size=40):
            super().__init__()
            self.make_chunks(source_dataset, chunk_size)

```

```

def make_chunks(self, source_dataset, chunk_size):
    all_x, all_y = [], []
    for x, y in source_dataset:
        for chunk in range(x.shape[0] // chunk_size):
            offset = chunk * chunk_size
            all_x.append(x[offset:offset + chunk_size])
            all_y.append(y)
    self.tensors = (torch.stack(all_x), torch.tensor(all_y))

chunked_data_train = ChunkedDataset(data_train_original)
chunked_data_val = ChunkedDataset(data_val_original)

# rebalance to compensate for any differences in length
chunked_data_train = balance_dataset(chunked_data_train)
chunked_data_val = balance_dataset(chunked_data_val)

print('chunked_data_train:', len(chunked_data_train))
print('chunked_data_val: ', len(chunked_data_val))

```

```

Samples per class: [33999, 4806]
- Repeating 29193 samples for class 1
- Repeating 24387 samples for class 1
- Repeating 19581 samples for class 1
- Repeating 14775 samples for class 1
- Repeating 9969 samples for class 1
- Repeating 5163 samples for class 1
- Repeating 357 samples for class 1
Samples per class: [8427, 1314]
- Repeating 7113 samples for class 1
- Repeating 5799 samples for class 1
- Repeating 4485 samples for class 1
- Repeating 3171 samples for class 1
- Repeating 1857 samples for class 1
- Repeating 543 samples for class 1
chunked_data_train: 67998
chunked_data_val: 16854

```

4.6 Preparing data loaders

As in the previous assignments, we will use the PyTorch [DataLoader](#) class to divide our datasets in minibatches.

(a) Run the code to create the data loaders. Look at the shape of the first minibatch.

```

In [ ]: batch_size = 192
        chunked_loaders = {
            'train': torch.utils.data.DataLoader(chunked_data_train, shuffle=True,
            'val': torch.utils.data.DataLoader(chunked_data_val, batch_size=batch_size)
        }

# print the x and y shapes for one minibatch
for (x, y) in chunked_loaders['train']:
    print(x.shape, y.shape)
    break

torch.Size([192, 40, 17]) torch.Size([192])

```

4.7 Implementing an LSTM (5 points)

Time series data such as the ECG recordings are a good target for recurrent neural networks (see [Section 9.4](#) of the D2L book).

The class below implements an RNN layer in PyTorch, using the equations discussed in [Section 9.4.2](#) of the book.

(a) Read through the code to see how the RNN works.

```
In [ ]: class RNN(torch.nn.Module):
    """RNN module.

    This implements an RNN module as discussed in Sections 9.4 and 9.5 of the
    D2L book (http://d2l.ai/chapter\_recurrent-neural-networks/rnn.html,
    http://d2l.ai/chapter\_recurrent-neural-networks/rnn-scratch.html).

    Parameters:
        num_inputs: scalar, the number of inputs to this module
        num_hiddens: scalar, the number of hidden units

    Input and output: see the forward function.
    """
    def __init__(self, num_inputs, num_hiddens):
        super().__init__()
        self.num_inputs = num_inputs
        self.num_hiddens = num_hiddens
        self.initialize_parameters()

    def initialize_parameters(self):
        """Initializes the parameters of the RNN module.

        This initializes the bias vector b_h and weight matrices W_xh and W_hh.
        """
        def triple():
            return (torch.nn.Parameter(torch.normal(0, 0.01, size=(self.num_inputs, self.num_hiddens))),
                    torch.nn.Parameter(torch.normal(0, 0.01, size=(self.num_hiddens, self.num_hiddens))),
                    torch.nn.Parameter(torch.zeros(size=(self.num_hiddens,))))

        # parameters for the rnn
        self.W_xh, self.W_hh, self.b_h = triple()

    def forward(self, inputs):
        """Computes the forward pass of the RNN module.

        Input:
            inputs: a tensor of shape (samples, steps, input features)
                    giving the input for each sample at each step

        Output:
            outputs: a tensor of shape (samples, steps, hidden features)
                     providing the hidden values at the end of each step
            state: a tuple (hiddens,)
                   the state of the RNN at the end of the last step,
                   with hiddens a tensor of shape (samples, hidden_features)
        """
        batch_size = inputs.shape[0]

        # initialize state
        state = (torch.zeros(size=(batch_size, self.num_hiddens),
                               dtype=inputs.dtype, device=inputs.device),)

        # run steps
        outputs = []
```

```

    for step in range(inputs.shape[1]):
        state = self.one_step(inputs[:, step], state)
        outputs.append(state[0])

    # concatenate outputs
    outputs = torch.stack(outputs, axis=1)
    return outputs, state

def one_step(self, x, state):
    """Run a single step of the RNN module.

    Input:
        x:      a tensor of shape (samples, input features)
                giving the input for each sample at the current step
        state: a tuple (hiddens,)
                the state of the RNN at the end of the previous step,
                with hiddens a tensor of shape (samples, hidden_features)
    """
    # extract current state
    (h,) = state

    # see http://d2l.ai/chapter_recurrent-neural-networks/rnn-scratch.h

    # new hidden
    h = torch.tanh(torch.mm(x, self.W_xh) + torch.mm(h, self.W_hh) + se

    # return the state
    return (h,)

def __repr__(self):
    return ('RNN(num_inputs=%d, num_hiddens=%d)' %
            (self.num_inputs, self.num_hiddens))

# quick sanity check
rnn = RNN(3, 5)
print(rnn)
print('Parameters:')
for name, param in rnn.named_parameters():
    print(' - %s:' % name, tuple(param.shape))

```

```

RNN(num_inputs=3, num_hiddens=5)
Parameters:
  - W_xh: (3, 5)
  - W_hh: (5, 5)
  - b_h: (5,)

```

The design of the LSTM module is more complex than that of the RNN, but it follows a similar pattern of looping over all steps in the input. You can use the RNN implementation as a basis for an LSTM module.

(b) Implement the LSTM module below.

(5 points)

The equations and code in [Section 10.1](#) can provide some inspiration. Be aware that the book uses (steps, samples, ...) instead of (samples, steps, ...) as the shapes for the input and output variables, so you probably cannot copy code directly. Use the RNN implementation above and adapt this to the LSTM equations from the book.

```

In [ ]: class LSTM(torch.nn.Module):
        def __init__(self, num_inputs, num_hiddens):
            super().__init__()

```

```

self.num_inputs = num_inputs
self.num_hiddens = num_hiddens
self.initialize_parameters()

def initialize_parameters(self):
    # initialize the LSTM
    # we have weights of input, forget, output gates and bias
    def triple():
        return (torch.nn.Parameter(torch.normal(0, 0.01, size=(self.num_inputs, self.num_hiddens))),
                torch.nn.Parameter(torch.normal(0, 0.01, size=(self.num_hiddens, self.num_hiddens))),
                torch.nn.Parameter(torch.zeros(size=(self.num_hiddens, 1))))

    # parameters for the LSTM
    self.W_xi, self.W_hi, self.b_i = triple() # Input gate
    self.W_xf, self.W_hf, self.b_f = triple() # Forget gate
    self.W_xo, self.W_ho, self.b_o = triple() # Output gate
    self.W_xc, self.W_hc, self.b_c = triple() # Input node

def forward(self, inputs):
    # implement the forward pass of the LSTM
    batch_size = inputs.shape[0]

    # initialize state (our state now has 2 components)
    state = (torch.zeros(size=(batch_size, self.num_hiddens),
                           dtype=inputs.dtype, device=inputs.device),
             torch.zeros(size=(batch_size, self.num_hiddens),
                           dtype=inputs.dtype, device=inputs.device))

    # run steps
    outputs = []
    for step in range(inputs.shape[1]):
        state = self.one_step(inputs[:, step], state)
        outputs.append(state[0])

    # concatenate outputs
    outputs = torch.stack(outputs, axis=1)
    return outputs, state

def one_step(self, x, state):
    # in this case, we need to define h and c (hidden and input node )
    (h, c) = state

    i_gate = torch.sigmoid(torch.mm(x, self.W_xi) + torch.mm(h, self.W_hi) + self.b_i)
    f_gate = torch.sigmoid(torch.mm(x, self.W_xf) + torch.mm(h, self.W_hf) + self.b_f)
    o_gate = torch.sigmoid(torch.mm(x, self.W_xo) + torch.mm(h, self.W_ho) + self.b_o)
    c_tilde = torch.tanh(torch.mm(x, self.W_xc) + torch.mm(h, self.W_hc) + self.b_c)

    # new hidden and new memory cell
    c = f_gate * c + i_gate * c_tilde
    h = o_gate * torch.tanh(c)

    # return the state
    return (h, c)

def __repr__(self):
    return ('LSTM(num_inputs=%d, num_hiddens=%d)' %
            (self.num_inputs, self.num_hiddens))

# quick sanity check
lstm = LSTM(3, 5)
print(lstm)

```

```
for name, param in lstm.named_parameters():
    print(' - %s:' % name, tuple(param.shape))
```

```
LSTM(num_inputs=3, num_hiddens=5)
- W_xi: (3, 5)
- W_hi: (5, 5)
- b_i: (5,)
- W_xf: (3, 5)
- W_hf: (5, 5)
- b_f: (5,)
- W_xo: (3, 5)
- W_ho: (5, 5)
- b_o: (5,)
- W_xc: (3, 5)
- W_hc: (5, 5)
- b_c: (5,)
```

4.8 Defining the training loop

As last week, we need to define some functions to run the train the models.

(a) Run the code to define the functions.

```
In [ ]: def accuracy(y_hat, y):
    # Computes the mean accuracy.
    # y_hat: raw network output (before sigmoid or softmax)
    #         shape (samples, classes)
    # y:      shape (samples)
    if y_hat.shape[1] == 1:
        # binary classification
        y_hat = (y_hat[:, 0] > 0).to(y.dtype)
    else:
        # multi-class classification
        y_hat = torch.argmax(y_hat, axis=1).to(y.dtype)
    correct = (y_hat == y).to(torch.float32)
    return torch.mean(correct)
```

```
In [ ]: def train(net, data_loaders, epochs=100, lr=0.01, device=d2l.try_gpu()):
    # Trains the model net with data from the data_loaders['train'] and dat
    net = net.to(device)

    optimizer = torch.optim.Adam(net.parameters(), lr=lr)

    animator = d2l.Animator(xlabel='epoch',
                           legend=['train loss', 'train acc', 'validation
                                   figsize=(10, 5))

    timer = {'train': d2l.Timer(), 'val': d2l.Timer()}

    for epoch in range(epochs):
        # monitor loss, accuracy, number of samples
        metrics = {'train': d2l.Accumulator(3), 'val': d2l.Accumulator(3)}

        for phase in ('train', 'val'):
            # switch network to train/eval mode
            net.train(phase == 'train')

            for i, (x, y) in enumerate(data_loaders[phase]):
                timer[phase].start()

                # move to device
                x = x.to(device)
```



```

y = y.to(device)

# compute prediction
y_hat = net(x)

if y_hat.shape[1] == 1:
    # compute binary cross-entropy loss
    loss = torch.nn.BCEWithLogitsLoss()(y_hat[:, 0], y.to(device))
else:
    # compute cross-entropy loss
    loss = torch.nn.CrossEntropyLoss()(y_hat, y)

if phase == 'train':
    # compute gradients and update weights
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

metrics[phase].add(loss * x.shape[0],
                   accuracy(y_hat, y) * x.shape[0],
                   x.shape[0])

timer[phase].stop()

animator.add(epoch + 1,
             (metrics['train'][0] / metrics['train'][2],
              metrics['train'][1] / metrics['train'][2],
              metrics['val'][0] / metrics['val'][2],
              metrics['val'][1] / metrics['val'][2]))

train_loss = metrics['train'][0] / metrics['train'][2]
train_acc = metrics['train'][1] / metrics['train'][2]
val_loss = metrics['val'][0] / metrics['val'][2]
val_acc = metrics['val'][1] / metrics['val'][2]
examples_per_sec = metrics['train'][2] * epochs / timer['train'].sum()

print(f'train loss {train_loss:.3f}, train acc {train_acc:.3f}, '
      f'val loss {val_loss:.3f}, val acc {val_acc:.3f}')
print(f'{examples_per_sec:.1f} examples/sec '
      f'on {str(device)}')
```

4.9 Constructing some networks (5 points)

In the next experiments you will train different network architectures to see how they perform on the ECG dataset.

The input to all networks has the shape (samples, time steps, features) = (mb_size, 40, 17). The output should be a single feature, shape (mb_size, 1), that will be used in a binary cross-entropy loss function. (The networks should not include the final sigmoid activation function.)

Some simple baselines:

- **FullyConnectedNet** : A simple fully connected network that takes all features.
- **MeanSpectrumNet** : A fully connected network that works on the mean spectrum over all time steps.

A convolutional network:

- **ConvNet** : This network does a convolution over the time steps, using the 17 input features as channels.

Some recurrent models:

- **RNNNet** : A recurrent network with a simple RNN module.
- **LSTMNet** : A recurrent network with a more advanced LSTM module.
- **TorchLSTMNet** : The same model, but using the PyTorch implementation of the LSTM.

FullyConnectedNet

(a) Check the implementation of the following baseline architecture:

- Linear layer: network inputs to 512 units followed by a ReLU.
- Linear layer: 512 to 256 units followed by a ReLU.
- Linear layer: 256 to the network output.

```
In [ ]: class FullyConnectedNet(torch.nn.Module):
    def __init__(self, inputs, outputs=1):
        super().__init__()

        # by defining these layers here, they are included in the
        # parameters() list of this module, so they can be trained
        self.linear = torch.nn.Sequential(
            torch.nn.Flatten(),
            torch.nn.Linear(inputs, 512),
            torch.nn.ReLU(),
            torch.nn.Linear(512, 256),
            torch.nn.ReLU(),
            torch.nn.Linear(256, outputs)
        )

    def forward(self, x):
        # x shape: (samples, steps, inputs)
        return self.linear(x)

net = FullyConnectedNet(40 * 17)
print(net)
```

```
FullyConnectedNet(
  (linear): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=680, out_features=512, bias=True)
    (2): ReLU()
    (3): Linear(in_features=512, out_features=256, bias=True)
    (4): ReLU()
    (5): Linear(in_features=256, out_features=1, bias=True)
  )
)
```

MeanSpectrumNet

(b) Check the implementation of the following baseline architecture:

- Compute the mean spectrum (mean over the steps dimension).
- Linear layer: network inputs to 128 units followed by a ReLU.

- Linear layer: 128 to 64 units followed by a ReLU.
- Linear layer: 64 to the network output.

```
In [ ]: class MeanSpectrumNet(torch.nn.Module):
    def __init__(self, inputs=17, outputs=1):
        super().__init__()

        self.net = torch.nn.Sequential(
            torch.nn.Linear(inputs, 128),
            torch.nn.ReLU(),
            torch.nn.Linear(128, 64),
            torch.nn.ReLU(),
            torch.nn.Linear(64, outputs),
        )

    def forward(self, x):
        # x shape: (samples, steps, inputs)
        # compute the mean over all steps
        x = torch.mean(x, axis=1)
        return self.net(x)

net = MeanSpectrumNet()
print(net)

MeanSpectrumNet(
  (net): Sequential(
    (0): Linear(in_features=17, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=64, bias=True)
    (3): ReLU()
    (4): Linear(in_features=64, out_features=1, bias=True)
  )
)
```

ConvNet

(c) Complete the implementation of the following architecture:

(1 point)

Convolution over the steps, using frequencies as channels:

- 1D-convolution: network inputs to 32 channels, kernel size 3, ReLU.
- Average pooling: 2.
- 1D-convolution: 32 to 64 channels, kernel size 3, ReLU.
- Average pooling: 2.
- 1D-convolution: 64 to 128 channels, kernel size 3, ReLU.
- `AdaptiveAvgPool1d(1)` : Compute the mean for each channel over all steps.
- Flatten.
- Linear layer: 128 to the network output.

```
In [ ]: class ConvNet(torch.nn.Module):
    def __init__(self, inputs=1, outputs=1):
        super().__init__()

        self.net = torch.nn.Sequential(
            # add the convolutional and pooling layers
            torch.nn.Conv1d(inputs, 32, kernel_size=3),
            torch.nn.ReLU(),
            torch.nn.AvgPool1d(2),
```

```

        torch.nn.Conv1d(32, 64, kernel_size=3),
        torch.nn.ReLU(),
        torch.nn.AvgPool1d(2),
        torch.nn.Conv1d(64, 128, kernel_size=3),

        torch.nn.AdaptiveAvgPool1d(1),
        torch.nn.Flatten(),
        torch.nn.Linear(128, outputs),
    )

    def forward(self, x):
        # x shape: (samples, steps, inputs)
        # swap the steps and inputs dimensions, so we can convolve over
        # the steps use the frequencies as channels
        x = x.transpose(2, 1)
        return self.net(x)

net = ConvNet()

```

RNNNet

(d) Check the implementation of the following architecture:

- RNN: network input to 128 hidden units.
- Use the final hidden state from the RNN.
- Linear layer: 128 to 128 units followed by a ReLU.
- Linear layer: 128 to the network output.

```

In [ ]: class RNNNet(torch.nn.Module):
    def __init__(self, inputs=17, outputs=1):
        super().__init__()

        self.rnn = RNN(inputs, 128)
        self.linear = torch.nn.Sequential(
            torch.nn.Linear(128, 128),
            torch.nn.ReLU(),
            torch.nn.Linear(128, outputs)
        )

    def forward(self, x):
        # x shape: (samples, steps, inputs)
        out, (h,) = self.rnn(x)

        # use the final RNN hidden state as input
        # for the fully connected part
        return self.linear(h)

net = RNNNet()
print(net)

```

```

RNNNet(
  (rnn): RNN(num_inputs=17, num_hiddens=128)
  (linear): Sequential(
    (0): Linear(in_features=128, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=1, bias=True)
  )
)

```

LSTMNet

(e) Implement the following architecture: (see RNNNet for an example)**(2 points)**

- LSTM: network input to 128 hidden units.
- Use the final hidden state from the LSTM.
- Linear layer: 128 to 128 units followed by a ReLU.
- Linear layer: 128 to the network output.

```
In [ ]: class LSTMNet(torch.nn.Module):
    def __init__(self, inputs=17, outputs=1):
        super().__init__()

        # define the LSTM layer and the linear network
        # (see RNNNet for an example)
        self.lstm = LSTM(inputs, 128)
        self.linear = torch.nn.Sequential(
            torch.nn.Linear(128, 128),
            torch.nn.ReLU(),
            torch.nn.Linear(128, outputs)
        )
    def forward(self, x):
        # x shape: (samples, steps, inputs)
        # call the LSTM layer and then the linear network
        out, (h,c) = self.lstm(x)
        return self.linear(h)

net = LSTMNet()
print(net)
```

```
LSTMNet(
  (lstm): LSTM(num_inputs=17, num_hiddens=128)
  (linear): Sequential(
    (0): Linear(in_features=128, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=1, bias=True)
  )
)
```

TorchLSTMNet

Implementing your own modules can be fun and good learning experience, but it is not always the most efficient solution. The built-in [LSTM implementation](#) from PyTorch is much faster than our own version.

(f) Implement a network similar to LSTMNet using the PyTorch torch.nn.LSTM module.**(2 points)**

```
In [ ]: class TorchLSTMNet(torch.nn.Module):
    # make this identical to LSTMNet, but use torch.nn.LSTM
    # instead of the LSTM layer you implemented yourself
    def __init__(self, inputs=17, outputs=1):
        super().__init__()

        self.lstm = torch.nn.LSTM(inputs, 128, batch_first=True)
        self.linear = torch.nn.Sequential(
            torch.nn.Linear(128, 128),
            torch.nn.ReLU(),
            torch.nn.Linear(128, outputs)
        )
    def forward(self, x):
```

```

# x shape: (samples, steps, inputs)
# call the LSTM layer and then the linear network
out, (h,c) = self.lstm(x)
# torch.squeeze(self.linear(h), 0)
return self.linear(h[0])

```

```

net = TorchLSTMNet()
print(net)

```

```

TorchLSTMNet(
  (lstm): LSTM(17, 128, batch_first=True)
  (linear): Sequential(
    (0): Linear(in_features=128, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=1, bias=True)
  )
)

```

4.10 Experiments

(a) Train the models on the chunked dataset.

Some of the outputs of the graphs are set to an ylim of (0,1.5) in order to see a better behavior of the training process.

```

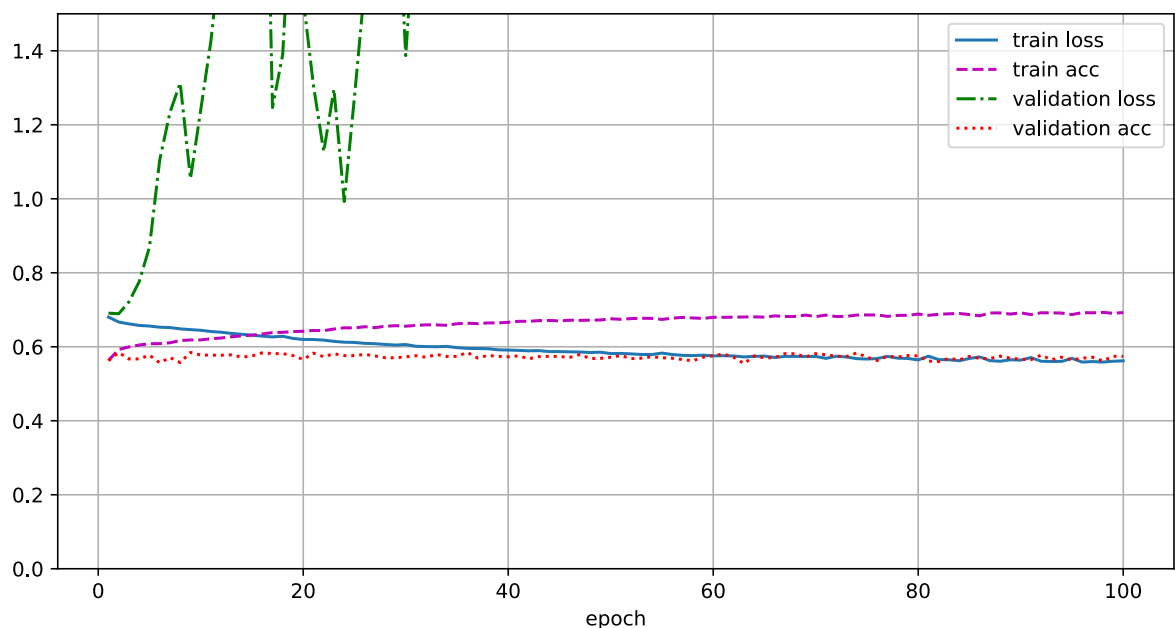
In [ ]: train(MeanSpectrumNet(), chunked_loaders, epochs=100, lr=0.01)
        plt.ylim(0,1.5)

```

```

train loss 0.562, train acc 0.692, val loss 4.254, val acc 0.574
89326.9 examples/sec on cpu
Out[ ]: (0.0, 1.5)

```



```

In [ ]: train(FullyConnectedNet(40 * 17), chunked_loaders, epochs=25, lr=0.01)
        plt.ylim(0,1.5)

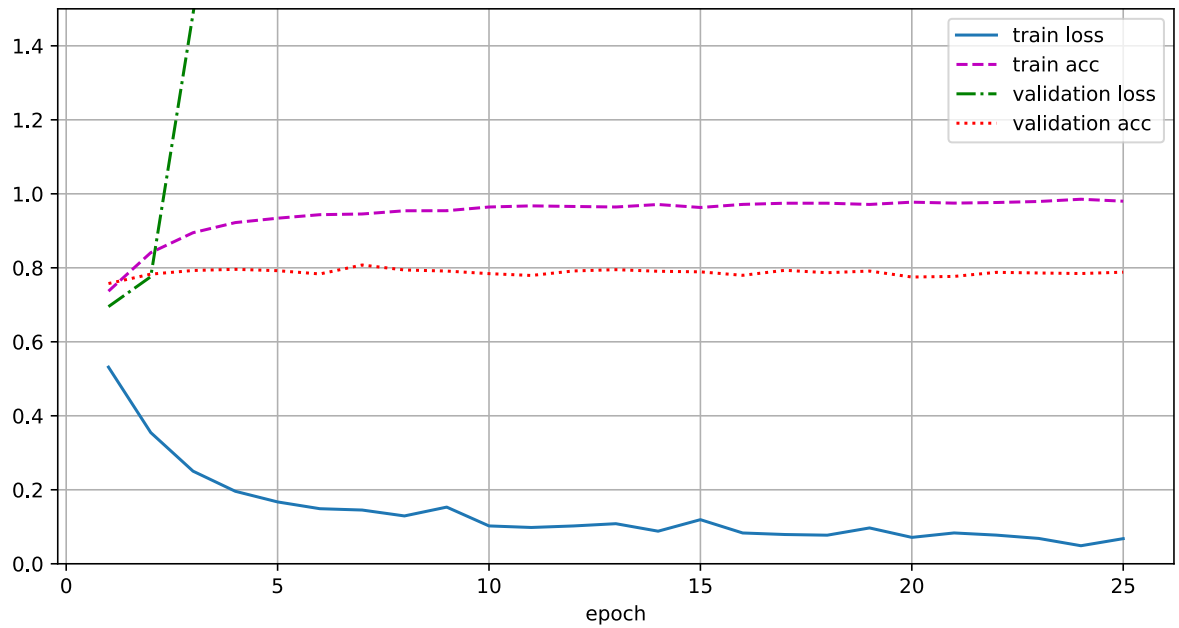
```

```

train loss 0.068, train acc 0.980, val loss 19.313, val acc 0.788
13876.9 examples/sec on cpu

```

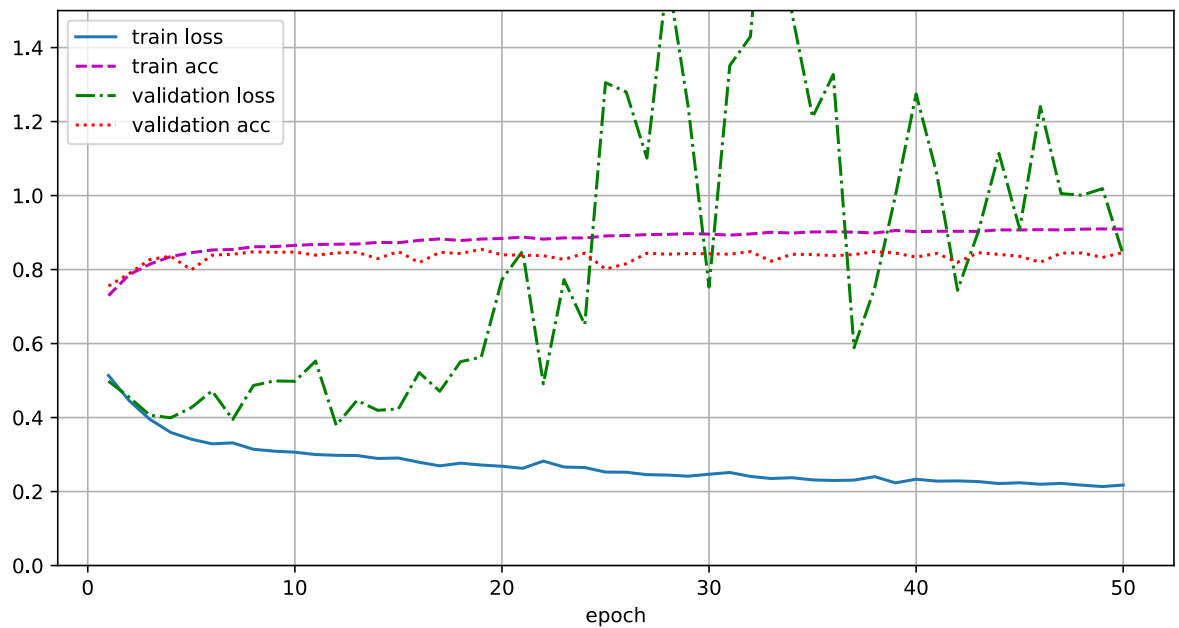
Out[]: (0.0, 1.5)



```
In [ ]: train(ConvNet(17), chunked_loaders, epochs=50, lr=0.01)
plt.ylim(0,1.5)
```

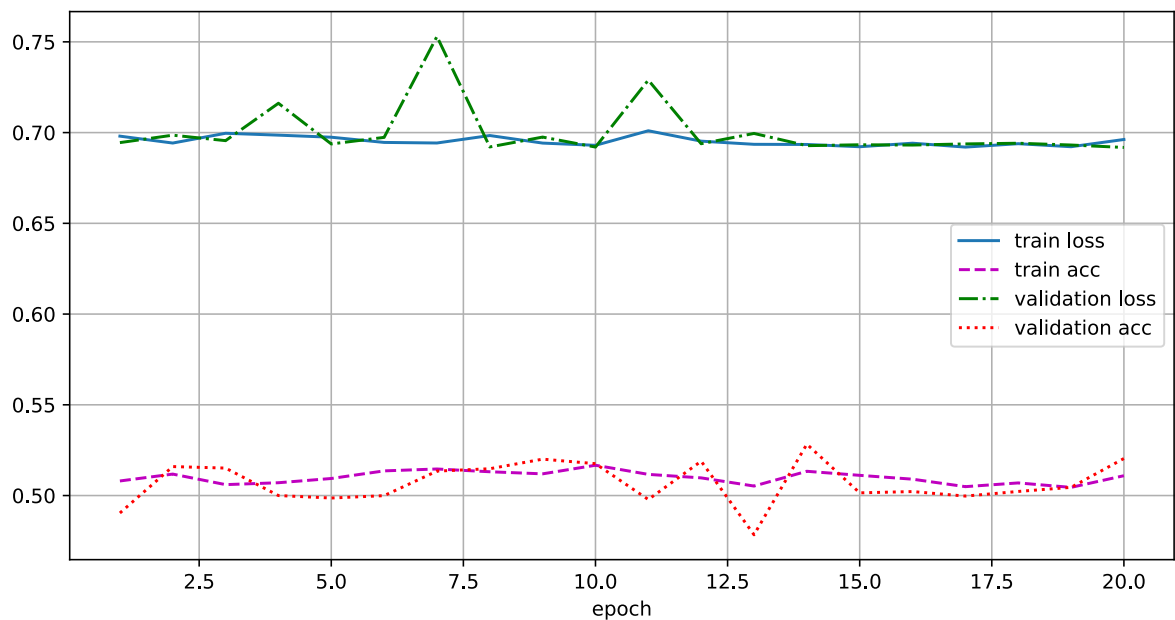
train loss 0.218, train acc 0.909, val loss 0.839, val acc 0.846
7807.0 examples/sec on cpu

Out[]: (0.0, 1.5)



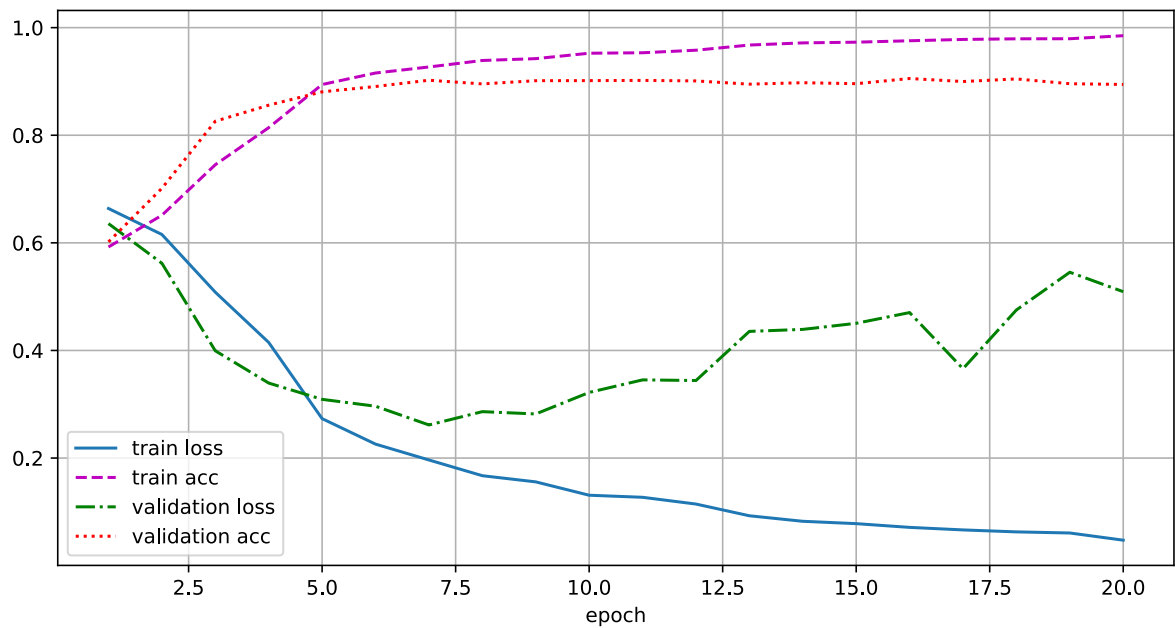
```
In [ ]: train(RNNNet(17), chunked_loaders, epochs=20, lr=0.01)
```

train loss 0.696, train acc 0.511, val loss 0.692, val acc 0.520
2145.9 examples/sec on cpu



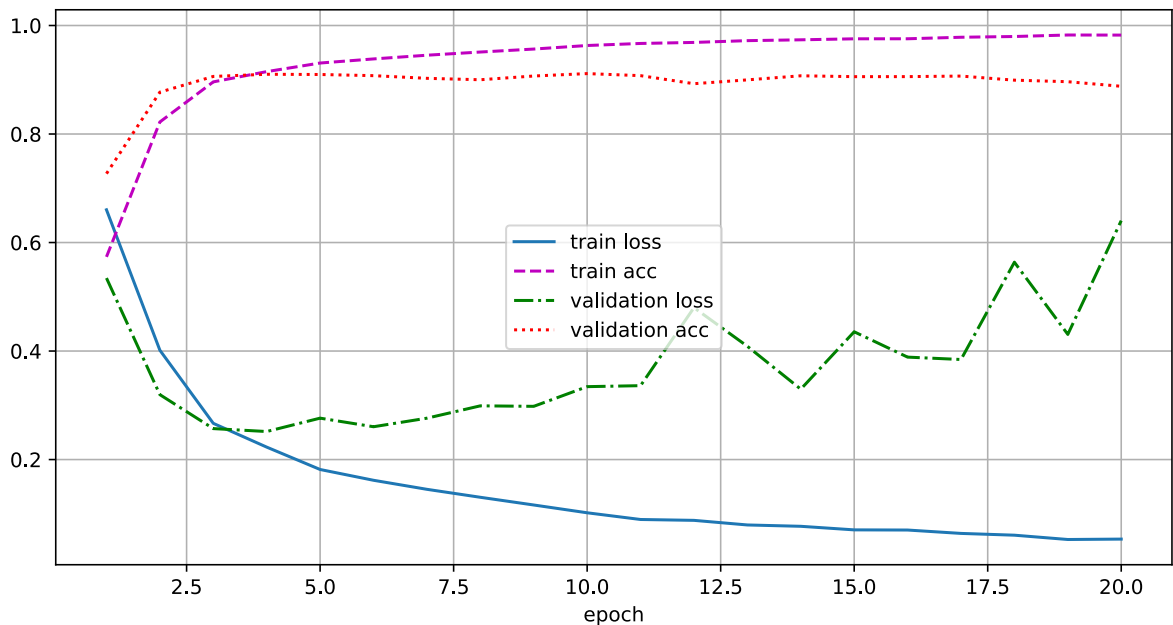
```
In [ ]: train(LSTMNet(17), chunked_loaders, epochs=20, lr=0.01)
```

train loss 0.047, train acc 0.985, val loss 0.509, val acc 0.894
1494.3 examples/sec on cpu



```
In [ ]: train(TorchLSTMNet(17), chunked_loaders, epochs=20, lr=0.01)
```

train loss 0.053, train acc 0.982, val loss 0.640, val acc 0.888
1298.2 examples/sec on cpu



4.11 Discussion (11 points)

(a) Briefly discuss and compare the performance of the models in your experiments.

Which worked best and why?

(2 points)

- *MeanSpectrumNet*: This model has a bad performance (slightly better than random). This is because it is a simple model that only computes the mean spectrum and then it is passed through a fully connected network. Therefore, it is not able to capture the temporal information of the ECG recordings.
- *FullyConnectedNet*: According to the simplicity of the model, it has a good performance. However, this model is overfitting with a high validation loss. This is because it is a simple model that only uses fully connected layers. Therefore, it is not able to capture the temporal information of the ECG recordings.
- *ConvNet*: This model has a good performance. This is because it is a more complex model that uses convolutional layers.
- *RNNNet*: The performance of this model is equivalent to random. Although it is a more complex model, it is no learn from the data.
- *(Torch)LSTMNet*: LSTM models are able to obtain good results. They both obtain similar performance, although the first one is slower than the version implemented by pytorch.

(b) Why do some of those models generalize better than others?

(2 points)

Because they have different structure and therefore they learn by using different strategies, that can be more or less accurate depending on the type of data that we are dealing with. This leads to have models that are unable to generalize because of their simplicity (Mean SpectrumNet), models that overfit(Fully ConnectedNet), models that are too complex (RNNNet) and models that are able to generalize well (LSTMs).

(c) How does your LSTM implementation compare with the PyTorch implementation? (1 point)

Both implementations obtain similar results. However, the pytorch implementation is faster than the one implemented by us. This makes sense as the pytorch implementation is optimized to be faster.

(d) Your RNN model probably didn't work well. Why is that model more difficult to train than the LSTM? (1 point)

The RNN model is more difficult to train than the LSTM because the RNN model is not able to capture the long-term dependencies. This is because the RNN model suffers from the vanishing gradient problem, where the gradients become smaller and smaller as the model is trained. Therefore, the model is not able to learn from the data. On the other hand, as the LSTM model uses gates that control the flow of information, it is able to handle this vanishing gradient issue. Thus, the LSTM model is able to learn from the data.

(e) The convolutional network and the LSTM in these experiments both work on the time dimension. What is an advantage of the convolutional network over the LSTM? (1 points)

As ConvNet uses parallel processing, it is faster than LSTM that uses sequential processing. Therefore, ConvNet is more efficient than LSTM.

Another advantage is that the structure of ConvNet is simpler than the LSTM model. Thus, it requires fewer parameters making it less prone to overfitting.

(f) What is an advantage of the LSTM over a convolutional network? (1 points)

ConvNet is able to capture the local dependencies using a kernel that slides over the data, being able to extract these relations. On the other hand, the LSTM is able to capture the long-term dependencies of the data. And as the ECG recordings are a time series data, the LSTM is able to perform slightly better.

(g) For reasons of speed, we used a fairly small window of 40 time steps. Suppose that we would make this window much larger. How do you think this would affect each model? (2 points)

- Models that would be affected the most: FullyConnectedNet (this would lead to more parameters and thus more chances to overfit), MeanSpectrumNet (harder to extract the information), RNNNet (harder to capture the long-term dependencies as the vanishing gradient problem would be more severe)

- Models that would be affected the least: LSTMs and ConvNet

(h) One of the difficulties with recurrent networks is that inputs from early steps are quite far away from the final result. How would you suggest to reduce that problem?

(1 point)

A potential solution for this could be adding the bidirectional parameter to the LSTM.

4.12 Data augmentation

Especially if your dataset is small, data augmentation can help to improve the performance of your network.

We have an easy way to add some data augmentation to the ECG dataset. In our preprocessing, we divided each recording into small chunks of 40 time steps, which we then reused in every epoch. We can add more variation to the training set by creating chunks at random positions.

The `DataLoader` class in PyTorch has a `collate_fn` parameter to which we can pass a function. This function is called for each minibatch in each epoch. We will use this to extract a random chunk from each sample.

The function `random_chunk_collate_fn` takes a minibatch of samples, chooses a random offset for each sample, extracts a small chunk at that position, and then concatenates and returns the result.

```
In [ ]: def random_chunk_collate_fn(samples, window_size):
        # Take a list of tensors of (steps_i, features),
        # extract a random window of length window_size from each tensor,
        # concatenate to a tensor of shape (samples, window_size, features).
        x_batch = torch.empty((len(samples), window_size) + samples[0][0].shape,
                               device=samples[0][0].device, dtype=samples[0][0].dtype)
        y_batch = torch.empty((len(samples),),
                               device=samples[0][1].device, dtype=samples[0][1].dtype)
        for i, (x, y) in enumerate(samples):
            # extract a random window
            offset = torch.randint(x.shape[0] - window_size, (1,))
            x_batch[i, :] = x[offset:offset + window_size]
            y_batch[i] = y
        return x_batch, y_batch
```

We construct a new DataLoader for our training set:

```
In [ ]: # test to see the x and y shapes for one sample
        random_chunk_loaders = {
            'train': torch.utils.data.DataLoader(data_train, shuffle=True, batch_size=batch_size,
                                                  collate_fn=lambda s: random_chunk_collate_fn(s)),
            'val':   chunked_loaders['val']
        }
        for (x, y) in random_chunk_loaders['train']:
            print(x.shape, y.shape)
            break
```

```
torch.Size([192, 40, 17]) torch.Size([192])
```

Observe that the pre-chunked dataset was much larger than the new dataset with on-the-fly chunking. You might want to increase the number of training epochs a bit to make sure that the network sees a similar number of examples.

```
In [ ]: print('Minibatches in chunked_loader[\'train\']: ', len(chunked_loader))
print('Minibatches in random_chunk_loaders[\'train\']: ', len(random_chunk_loader))

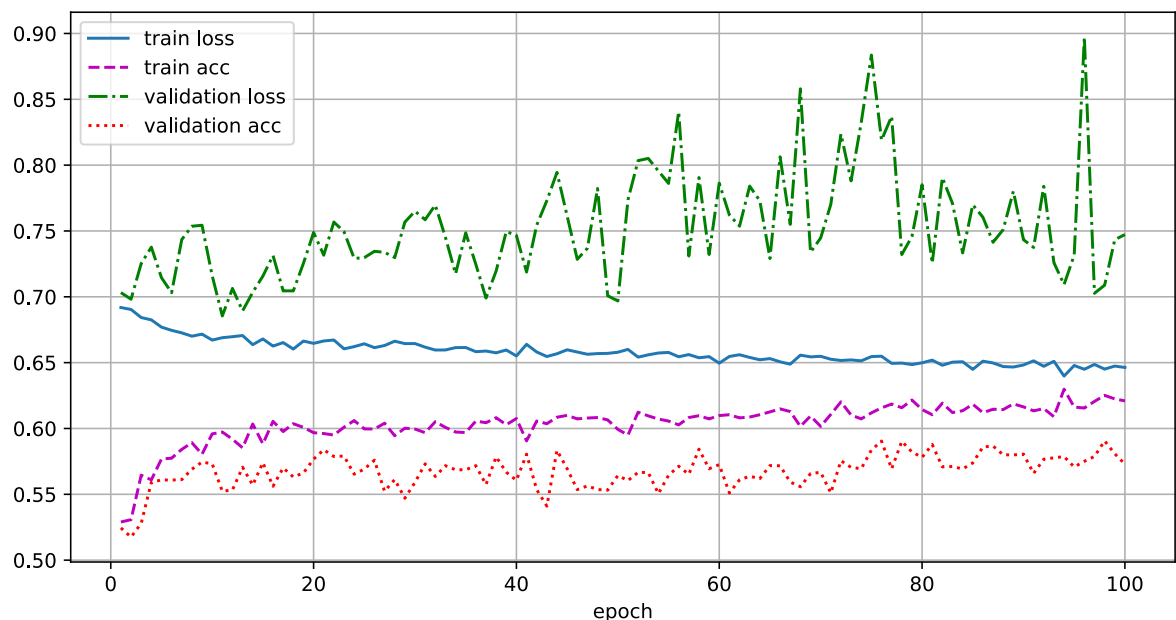
Minibatches in chunked_loader[\'train\']: 355
Minibatches in random_chunk_loaders[\'train\']: 42
```

Let's see how this data augmentation method affects the performance of your networks.

(a) Train the MeanSpectrumNet, FullyConnectedNet, ConvNet and TorchLSTMNet from the previous experiments on data from the `random_chunk_loaders`.

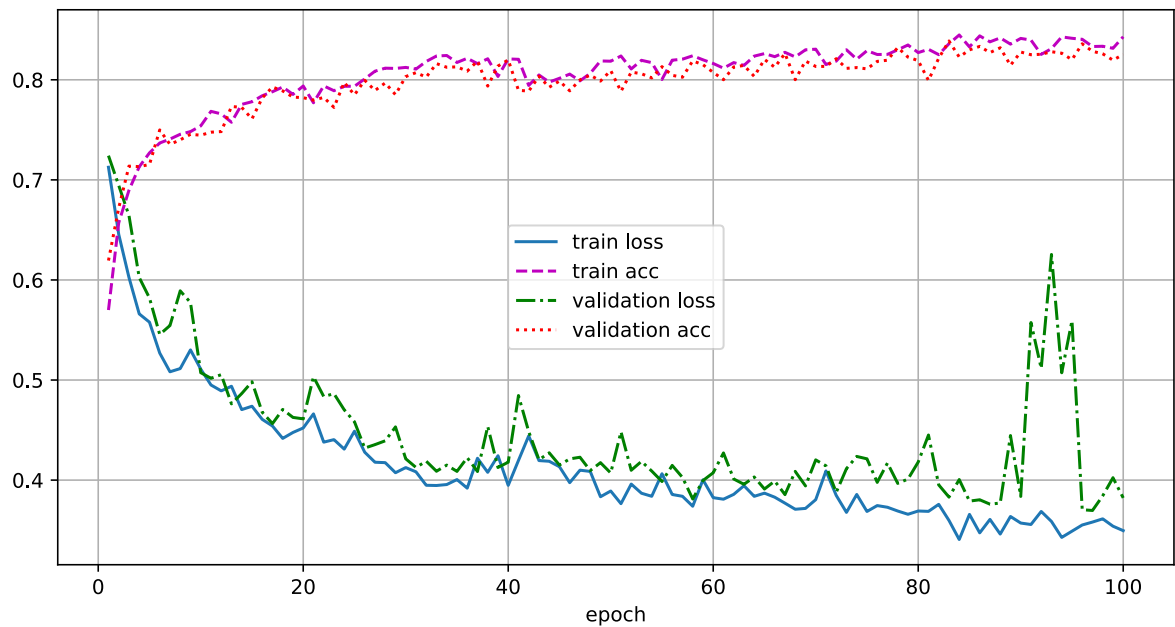
```
In [ ]: train(MeanSpectrumNet(), random_chunk_loaders, epochs=100, lr=0.01)
```

train loss 0.646, train acc 0.621, val loss 0.747, val acc 0.573
77942.3 examples/sec on cpu



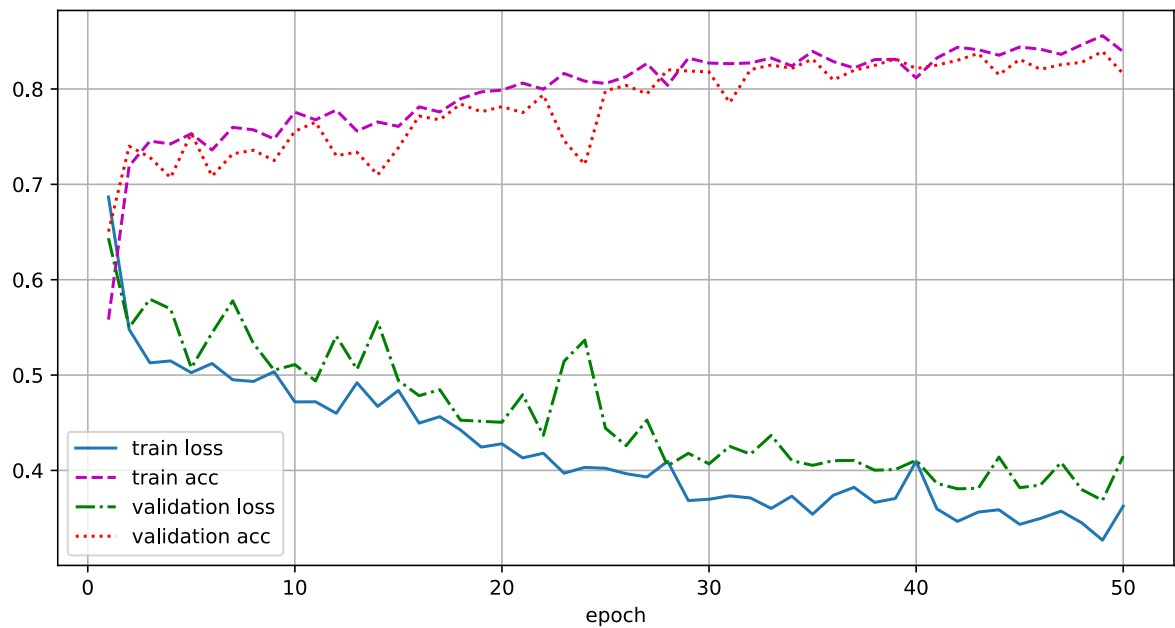
```
In [ ]: train(FullyConnectedNet(40 * 17), random_chunk_loaders, epochs=100, lr=0.01)
```

train loss 0.350, train acc 0.843, val loss 0.382, val acc 0.825
13843.1 examples/sec on cpu



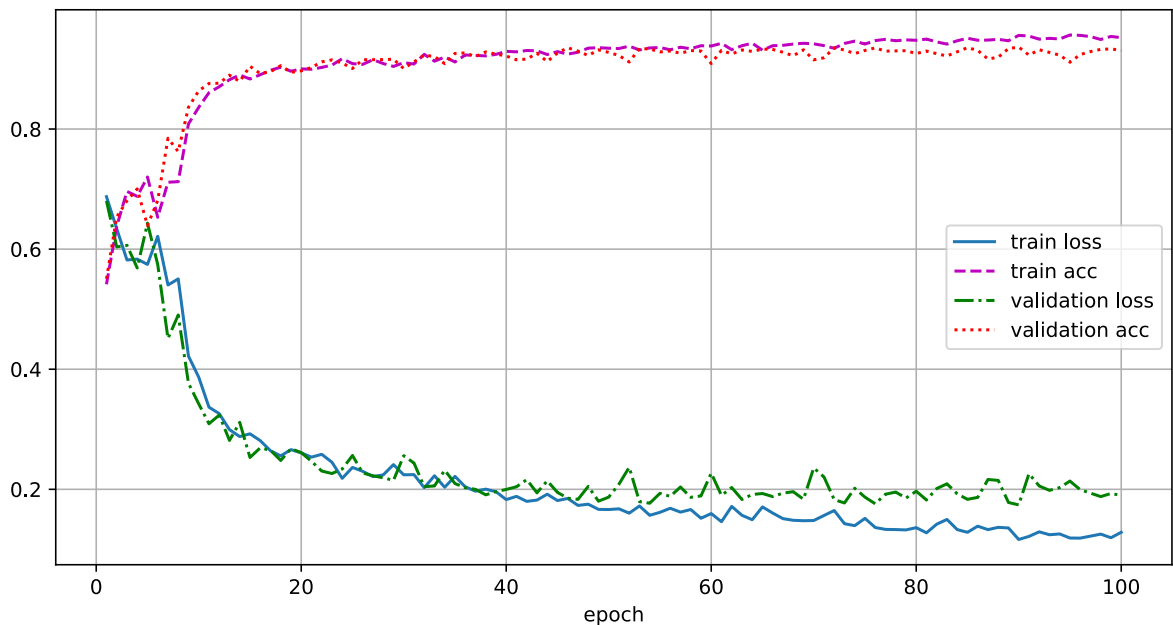
```
In [ ]: train(ConvNet(17), random_chunk_loaders, epochs=50, lr=0.01)
```

train loss 0.363, train acc 0.839, val loss 0.415, val acc 0.816
7809.9 examples/sec on cpu



```
In [ ]: train(TorchLSTMNet(17), random_chunk_loaders, epochs=100, lr=0.01)
```

train loss 0.128, train acc 0.952, val loss 0.190, val acc 0.931
1303.0 examples/sec on cpu



4.13 Discussion (9 points)

(a) How does the data augmentation influence the training and validation results?

Can you explain this?

(2 points)

The effect of the data augmentation on the training results is that the training loss and training accuracy are worse than without. The explanation for this is that the model becomes better at generalizing because it overfits the data less.

This is also reflected in the validation results as these were mostly better for the used models:

MeanSpectrumNet: just a bit better, if not the same

FullyConnectedNetwork: better than non-augmented FCN and augmented MSN; also, the validation and train accuracy are much closer to each other

ConvNet: validation accuracy is not better, if not worse; however, the validation loss is much better than for the non-augmented, where the validation loss of the non-augmented CN was wildly fluctuating.

TorchLSTMNet: validation accuracy is somewhat better, much closer to the training accuracy; the validation loss is also much more stable and better than for the non-augmented network.

(b) Why does the data augmentation affect some models more than others? (1 point)

This is a natural consequence of the different types of architectures, and their accompanying structures and complexities: models that tend to overfit - like CN, FCN, and TLSTMN - will need more data to train appropriately, so this should also be reflected in their resulting validation accuracies and losses. From theory we know that CN and TLSTMN will have translational invariance built into their structure, such that more (augmented) training data will lead to better learning. Obviously, the FCN also benefits from more training data as

it gives more invariance as well, allowing for better learning. The MSN will not be affected that much, since the mean over the means of the spectra will remain the same.

(c) Should we also do data augmentation on the validation set? Why, or why not?

(1 point)

No. The augmentation should only be implemented in the training phase since the validation and test sets are supposed to consist of realistic data; if we add augmentations to these sets as well, it would skew the reality of 'real', to be tested, data, which defeats the purpose of using augmentations.

(d) Data augmentation is often a good way to add some domain knowledge to your model. Based on your knowledge of ECGs, why is (or isn't) our augmentation method a good idea?

(1 point)

For the application of deep learning to ECG data, using the implemented data augmentation correctly integrates domain knowledge, and so it is a good augmentation method. We know that ECG data are time series in which we are looking for patterns rather than a set structure, such that in this case looking at separate timeframes does not mess with the inherent structure of the data. Maintaining this structure is paramount to efficient data augmentation, making the used method good.

(e) Give an example of another suitable augmentation method and explain why it would work for this data.

(2 points)

Given we are working with ECGs, which reflect the heart rate, we could introduce slight changes in either amplitude (scaling) or time. These would be valid augmentation since they still maintain the inherent structure of the data; it is not weird that someone has a slightly stronger signal, or a faster one.

(f) Give an example of an augmentation method that might be suitable for other data but would probably not work here. Explain why.

(2 points)

For one's data it is important to keep in mind the specific arrangement of signal over time, to be able to extract the required patterns. An augmentation method such as flipping the data - commonly used in image DNNs - would most probably be a very bad augmentation, given that it does not correspond to the physiological structure of the data.

The end

Well done! Please double check the instructions at the top before you submit your results.

This assignment has 31 points.

Version 7bd5a19 / 2023-09-25